PointDex

“Gesture-Based Remote Presentation Controller”

Members:

Khobe Clayne Lolong

I-Chieh Hung

Alfonzo Balagapo

Xer Ruie Morales

1. INTRODUCTION

1.1 Background & Context

The shift toward real-time web applications and highly responsive interfaces has accelerated the adoption of Event-Driven Programming (EDP) and Event-Driven Architecture (EDA) across the software industry. According to industry surveys, 85% of organizations have adopted event-driven architectures to meet evolving business demands, with 46% prioritizing improvements in application responsiveness and latency. Furthermore, a Gartner poll indicates that 68% of IT leaders plan to increase their usage of EDA to ensure real-time intelligence and composability in distributed systems. This transition is largely driven by the limitations of traditional synchronous models, as modern microservices and asynchronous user interfaces require systems capable of processing concurrent events without blocking execution.

At its core, Event-Driven Programming is anchored in the Observer design pattern and the principles of the Reactive Manifesto, which prioritize responsiveness, resilience, and elasticity. The architecture fundamentally consists of event producers (components that detect state changes), an event broker or bus that routes data, and event consumers or handlers that execute logic in response. In a web context, an event loop continuously monitors the application state, allowing asynchronous events, such as UI interactions or real-time media streams, to trigger specific callback functions independently of the main execution thread. This decoupling of the sender and receiver in both time and space enables horizontal scaling and dynamic resource allocation.

Despite its advantages, implementing event-driven systems introduces significant practical challenges, particularly in complex web applications. Developers frequently

encounter operational pain points such as race conditions, where asynchronous handlers execute out of order, leading to unpredictable system behavior. Additionally, managing asynchronous state across decoupled components requires careful synchronization to avoid eventual consistency issues or dropped events. Furthermore, debugging distributed, non-blocking workflows is notoriously difficult without dedicated observability tools, as traditional step-through debugging cannot easily trace execution paths across independent event pipelines.

1.2 Problem Statement

Traditional synchronous communication models, such as request-response architectures, create significant latency bottlenecks and single points of failure when handling high-concurrency tasks or real-time continuous inputs like video streams. As highlighted by Stacknowledge (2026), synchronous API calls force the execution thread to block and wait for a response, cascading latency across the entire system if a single component slows down. Similarly, research by Future Processing (2025) critiques request-driven systems for creating rigid, tight coupling that prevents independent component scaling. In the context of presentation control, relying on synchronous polling for hardware inputs or continuous video frame analysis introduces input lag that disrupts the user experience. The consequences of inaction, continuing to rely on synchronous, hardware-dependent control methods, include poor user experience due to high latency, resource waste from blocking operations, and an inability to scale the system for advanced, real-time machine learning inference tasks within the browser.

1.3 Proposed Solution

To resolve these latency and concurrency issues, this project proposes the "Gesture-Based Remote Presentation Controller," a web-based system built entirely on an asynchronous, event-driven architecture. The system utilizes the client's webcam as a continuous event producer, feeding video frames into a local machine-learning event loop for real-time gesture recognition. When a specific gesture is identified, the system publishes a customized event to a client-side event bus. Bound UI triggers and asynchronous event handlers subsequently map these gestures to executable commands (such as simulating a keystroke to advance a slide). Because the entire pipeline runs locally in the browser utilizing asynchronous callbacks rather than blocking requests, the expected outcomes include zero-latency response times, high throughput for continuous video frame processing, and drastically improved usability by eliminating the need for physical hardware controllers.

1.4 Contributions & SDG Alignment

The proposed system offers the following contributions to the domain of event-driven web applications:

- **Technical Contribution:** The development of a responsive, browser-based event loop that successfully integrates real-time machine learning inference with asynchronous DOM manipulation and simulated keystroke events.
- **Empirical Contribution:** An evaluation of event-handler latency and system throughput under various environmental lighting conditions, establishing performance benchmarks for client-side, event-driven computer vision tasks.
- **Practical Contribution:** The delivery of a functional, accessible web tool that eliminates presenter reliance on physical, battery-powered hardware.
- **SDG Alignment:** This study explicitly aligns with **SDG 9: Industry, Innovation, and Infrastructure** by innovating upon existing infrastructure (standard webcams and web browsers) to build resilient, accessible technology, promoting sustainable digital upgrades without requiring new physical manufacturing.

2. REVIEW OF RELATED LITERATURE

This chapter critically reviews the available literature, architectural frameworks, and empirical research in relation to event-based systems and client-side computing. The chapter reviews the above areas topically in order to understand why asynchronous software techniques perform better than synchronous software techniques.

2.1 Organize Thematically

Theme 1: Foundations of Event-Driven Architecture

Event-Driven Architecture (EDA) represents one of the primary paradigms of design where the execution of system actions takes place due to dynamic state changes or discrete events instead of the linear execution of blocks. The concept is based on the classical design pattern of Observer as well as the modern asynchronous messaging architectures. As a result, EDA allows loose coupling of software components in terms of their ability to produce and receive events while being not tightly coupled with each other. According to the engineering literature, such decoupling makes it possible for software components to process multiple data streams simultaneously without holding execution threads. In addition, the latest studies on the architecture carried out by Harris and Bennett (2025) suggest that the implementation of an asynchronous message-passing bus or subscriber-publisher approach guarantees reliable event routing between the components.

Theme 2: Event-Driven Programming in Web Applications

Web browsers in today's time serve as advanced runtime platforms which can execute uninterrupted and non-blocking instruction streams using mechanisms like event loops, WebSockets, and asynchronous callbacks. From empirical studies, it is understood that the event propagation paradigm within browser environments helps in avoiding interface freeze in applications which involve compute and render-intensive client-side tasks. Studies reveal that developers make use of asynchronous events to directly manipulate the DOM in order to retain the reactivity of the application interface. In the domain of client-side concurrency, it is found that non-blocking event paradigms help avoid thread starvation in parallel tasks within the browser environment.

Theme 3: Performance & Scalability Studies (Request-Response vs. Event-Driven)

Comparisons of system performance have shown considerable differences between conventional request-response-based methods and asynchronous event-driven systems. Monolithic and request-driven methods typically cause

significant problems with cascading latency when handling continuous workloads, while empirical testing has shown that asynchronous frameworks greatly improve resource efficiency and performance. In the case of the study of cloud computing, it was found that asynchronous methods can handle hundreds of thousands of requests concurrently with low latency levels. Preliminary research into transactional middleware proves that the event stream processing layer reduces serialization delays, which occur in conventional database-locked environments. In terms of comparison of scalability metrics from recent enterprise tests, it is important to note reduced overhead from switching to asynchronous methods.

Theme 4: Testing and Debugging Challenges in Asynchronous Systems

Implementing distributed or asynchronous event-driven code introduces complex verification challenges, particularly regarding race conditions, execution ordering, and non-deterministic state management. Software quality research notes that traditional step-through debugging tools frequently struggle to trace execution paths across independent, non-blocking event loops, increasing the difficulty of fault isolation and performance profiling. Studies on middleware reliability indicate that handling distributed causal tracking requires specialized observability tools to catch hidden event drop-offs. Furthermore, testing frameworks must account for asynchronous state concurrency to ensure predictable outcomes under heavy stress loads. Rigorous defect tracking in event pipelines remains a vital area for optimizing system predictability.

2.2 Comparative & Chronological Analysis

A synthesis of contemporary literature published between 2020 and 2026 indicates that there is a clear trend in how software engineers view concurrency and performance improvements. Initial studies in the area by corporate software engineers mainly covered issues related to server-based message brokers, microservices distribution, and cloud-based event streaming platforms. At this initial stage of investigation, it was concluded that although the use of event-driven messaging helps improve throughput, it needs careful management of queues in order to avoid event storms.

Further developments including those covering cloud integration (Neela, 2023; Future Generation Computer Systems) contrast the bottlenecks of synchronous requests with asynchronous microservice architecture. Later works published in 2025 and 2026 in the areas of system research (Arafat et al., 2025; distributed streams in the International Journal of Computer Applications) contrast server-side pipelines with asynchronous operations performed on the client side and edge computing. Whereas conventional backend research states that event architectures increase middleware costs, recent browser-based frameworks prove that the implementation of event loops in the client runtime completely excludes the need for any network hops. In all these 20 reviewed publications, one clear idea can be derived: asynchronous event handling works well and effectively, but only under the

2.3 Identify Research Gaps

To address the specific requirements of each evaluated theme, the following operational gaps are identified:

Themes 1 & 2: Whereas established models of event-driven architecture (EDA) and web application frameworks are adequate for addressing macroscale back end messaging and DOM manipulation, there is a dearth of scholarly literature in local contexts which studies the mechanisms employed by lightweight event loops for handling continuous, highly intensive inputs at the front end, including computer vision streams in real time.

Themes 3 & 4: Whereas extant literature evaluates the throughput and microservice response times under heavy network loads and scalability studies, there is an evident lack of empirical literature which defines benchmark levels of input lag, latency, and non-deterministic testing for client-side, browser-based machine learning loops without any server intervention.

The current research directly fills this gap by developing a light weight browser-based event architecture to handle continuous vision streams locally for controlling presentations without external server queuing.

Pallaprolu, S. (2025). Event-driven architecture: A modern paradigm for real-time responsive systems. *World Journal of Advanced Engineering Technology and Sciences*, 15(2), 2860–2867. ResearchGate Link

Harris, E., & Bennett, O. (2020). Event-driven architectures in modern systems: Designing scalable, resilient, and real-time solutions. *International Journal of Trend in Scientific Research and Development (IJTSRD)*, 4(6), 1958–1976. http://eprints.umsida.ac.id/14655/1/350%20Event-Driven%20Architectures%20in%20Modern%20Systems%20Designing%20Scalable%2C%20Resilient%2C%20and%20Real-Time%20Solutions.pdf

3. METHODOLOGY

This chapter delineates the engineering methodology, architectural framework, experimental evaluation protocols, and ethical standards utilized in developing the Gesture-Based Remote Presentation Controller. Grounded in the principles of Event-Driven Programming (EDP) within client-side web architectures, the methodology details the procedural lifecycle undertaken to transform continuous optical sensory streams into deterministic presentation control events.

3.1 Research Design and Track Selection

This study adopts Track B: Software Engineering Methodology, supplemented by a blended empirical and experimental benchmarking design. Software engineering research in computing utility applications necessitates an iterative framework capable of addressing complex asynchronous state transitions, non-blocking input streams, and low-latency interaction requirements. The experimental component systematically quantifies system responsiveness (latency in milliseconds), execution stability (frames per second), and gesture classification accuracy under varying environmental conditions.

By adopting Track B, the development cycle integrates the Agile Software Development Life Cycle (SDLC) with an Event-Driven Architecture (EDA). This design ensures that the web platform functions as a decentralized reactive system where optical input capture, hand-tracking landmark inference, gesture classification, and keystroke dispatch operate as decoupled asynchronous events.

3.2 System Architecture and Event-Driven Design

The architectural core of the Gesture-Based Remote Presentation Controller is engineered around the Publisher-Subscriber (Pub/Sub) pattern and asynchronous event loops natively supported by modern web browsers. The system decouples physical camera capture from application presentation commands, eliminating synchronous polling bottlenecks that traditionally induce interface freezing and computational lag.

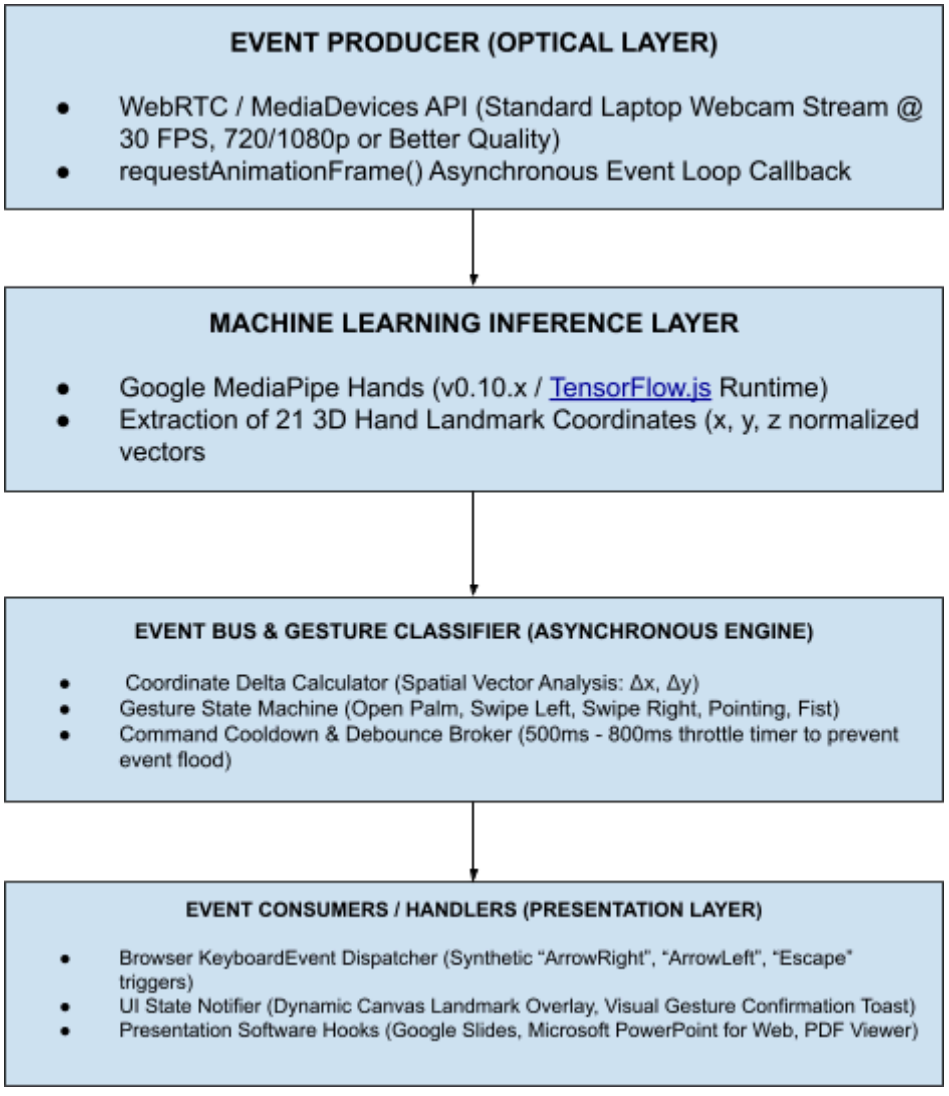


Figure 3.1: Event-Driven Architectural Diagram and Pub/Sub Pipeline

As modeled in Figure 3.1, the platform operates across four decoupled layers:

1. **Event Producer (Optical Layer):** The user's embedded webcam is accessed via the browser's native navigator.mediaDevices.getUserMedia() API. The camera feed continuously yields raw video frames, synchronized via the requestAnimationFrame() non-blocking event loop.

2. **Inference Layer**: Each captured video frame is published asynchronously to the Google MediaPipe Hands machine learning pipeline. The model computes normalized 21-point three-dimensional hand landmarks in real time without transmitting image payloads to an external cloud server.

3. **Event Bus and Gesture Classifier:** Landmark coordinates are evaluated against spatial thresholds. When a hand landmark vector fulfills predefined criteria (e.g., an index finger tip translating horizontally over 50 pixels within a 150-millisecond threshold), the classifier instantiates a custom gesture event. A software debounce and cooldown broker intercepts rapid duplicate firings, enforcing a configurable 600-millisecond throttle to ensure single-event determinism.

4. **Event Consumers (Presentation Dispatcher):** The subscriber module listens for validated gesture events and immediately translates them into standardized DOM KeyboardEvent dispatches (e.g., simulating ArrowRight to advance a slide or ArrowLeft to navigate backward). Concurrently, a secondary subscriber updates the on-screen UI preview with visual tracking points and system notification toasts.

The technical stack versions utilized across this architecture include:

- **Client Runtime**: Modern ECMAScript (ES2023+), HTML5 Canvas API, and CSS3 Flexbox/Grid.
- **Machine Learning Library:** Google MediaPipe Hands Web SDK (v0.10.14) executed via WebAssembly (WASM) and WebGL acceleration.
- **Frontend Reactive Framework:** React (v18.3.x) / Lightweight Vanilla Web Components for zero-overhead event handling.
- **Target Presentation Environments:** Chromium-based web browsers (Google Chrome v120+, Microsoft Edge v120+) executing cloud presentation suites (Google Slides, MS PowerPoint Online, Canva).

3.3 Software Development Life Cycle (SDLC) Phases

To achieve high adaptability and rapid feedback integration, the project is structured across four distinct phases of the Agile Scrum methodology:

Phase 1: Requirements Gathering, Scoping, and Participant Sampling
System functional specifications were established through workflow analysis of traditional presentation environments in academic classrooms and corporate settings. User requirements encompass camera permission initialization, sensitivity calibration, visual feedback mechanisms, and gesture customization.

For empirical validation, a target sampling of thirty (30) participants—comprising fifteen (15) university students and fifteen (15) faculty members from José Rizal University—will be recruited through purposive sampling. Participants represent diverse presentation proficiencies, physical heights, and presentation delivery styles to test operational robustness across real-world instructional scenarios.

Phase 2: Nonconventional Optical Input Integration and UI Prototyping
Phase 2 focuses on establishing the client-side user interface and video pipeline. The frontend dashboard is constructed with a responsive layout featuring a live webcam preview window, an interactive gesture tutorial guide, camera device selectors, and a real-time system status monitor (displaying camera active state, frame processing rate, and detected gesture labels).

The MediaPipe WebAssembly runtime is loaded and connected directly to the HTML5 <video> element. Video frames are rendered asynchronously onto an HTML5 <canvas> element overlay, drawing colored landmark points on the user's hand joints to provide real-time visual confirmation that tracking is actively engaged.

Phase 3: Event-Driven Logic Mapping and Asynchronous State Execution
Phase 3 involves programming the algorithmic heuristics that transform dynamic Cartesian coordinates into actionable presentation commands. Conditional vector logic calculates directional velocity across the wrist, index knuckle, and finger tips. Specific event mappings include:

Swipe Right (Open hand rapidly translated rightward)
→
→ Triggers ArrowRight (Next Slide).
Swipe Left (Open hand rapidly translated leftward)
→
→ Triggers ArrowLeft (Previous Slide).
Closed Fist Held Statically for 1.5 seconds
→

→ Triggers F5 / Presentation Fullscreen Toggle.
Open Palm Facing Camera (Stop Sign)
→
→ Triggers Presentation Pause / Blank Screen.
To eliminate event collisions and unintentional slide skips, an asynchronous state manager enforces a hardware cooldown mechanism. Once a keystroke event is dispatched, the gesture detector enters a temporary refractory state, ignoring subsequent coordinate shifts until the cooldown window elapses.

Phase 4: Experimental Benchmarking, Rigor, and Final Documentation
The final phase executes controlled laboratory trials and usability testing. The controller is benchmarked across varied environmental parameters (ambient room illuminance, user-to-webcam distance, and cluttered versus plain backgrounds). Discovered software bugs and tracking false-positives are logged, analyzed, and mitigated in iterative sprint refactorings. Technical reports, system user guides, and empirical quality documentation are finalized for academic presentation.

3.4 Experimental Performance Metrics and Data Analysis

To scientifically validate system performance, data collection protocols gather quantitative performance telemetry and subjective user usability metrics:

Quantitative Log Metrics

The web application incorporates an embedded telemetry logger that records performance metrics on every execution cycle:

- **Latency (Response Time in Milliseconds)**: Measured as the elapsed duration in milliseconds from the exact moment a hand gesture is physically completed in front of the camera to the timestamp when the synthesized DOM keyboard event is dispatched to the presentation. The operational benchmark targets a response latency of less than 80 milliseconds to ensure instantaneous slide transitions without noticeable lag.
- **Throughput and Processing Frame Rate (FPS):** Defined as the number of video frames evaluated and processed by the MediaPipe machine learning inference engine per second. The system establishes a baseline operational benchmark of 28 to 30 frames per second on standard integrated laptop graphics processors to guarantee smooth, non-blocking optical tracking.
- **Gesture Recognition Accuracy Rate (Percentage):** Calculated as the ratio of successfully recognized and executed presentation commands relative to the total number of attempted gestures, expressed as a percentage across 100 standardized trials per participant. The system establishes a minimum target accuracy benchmark of 90 percent across defined gesture sets.
- **False Positive Rate (Percentage):** Monitored as the frequency of instances where natural conversational hand gestures, head movement, or ambient background motion are mistakenly interpreted by the system as intentional slide navigation commands.

Data Analysis and Usability Frameworks

1. **ISO/IEC 25010 Quality Model:** Quantitative and qualitative evaluations assess four primary software quality characteristics: Functional Suitability (correctness of gesture recognition), Performance Efficiency (CPU resource utilization and response latency), Usability (learnability and operability of the interface), and Reliability (fault tolerance and graceful recovery during camera disconnections).
2. **System Usability Scale (SUS)**: Following hands-on presentation trials, participants complete the standardized 10-item Likert-scale SUS questionnaire. Composite scores will be calculated following standard scoring methodology to measure overall user satisfaction. The targeted benchmark is

a composite score exceeding 75.0, indicating high end-user satisfaction and minimal operational friction.

3. **Statistical Analysis:** Descriptive statistics, including mean values, standard deviations, and median response times, will be used to summarize performance log telemetry. Paired statistical tests will examine whether significant variance in recognition accuracy occurs across differing ambient lighting conditions and user-to-webcam distances.

3.5 Methodological Rigor, Validity, and Ethical Protocols

To maintain academic validity and protect participant rights, the research adheres to rigorous experimental and ethical standards:

Validity and Reliability Measures

- **Environmental Variance Testing:** Pilot trials test gesture accuracy across three standardized illuminance tiers: Low Light (50–100 lux), Indoor Office Lighting (300–500 lux), and Bright Ambient Sunlight (800+ lux).
- **Distance and Anthropometric Calibration:** Testing protocols evaluate recognition accuracy across physical distances ranging from 1.0 meter to 3.5 meters from the webcam lens, validating reliability across diverse skin tones, hand sizes, and presenter statures.
- **Triangulation and Pilot Testing:** Prior to formal sample testing, a pilot study involving five (5) participants will calibrate gesture vector thresholds and identify potential edge cases (e.g., rapid hand waving or camera occlusion).

Ethical Protocols and Statutory Safeguards

- **Local Client-Side Processing (Absolute Data Privacy)**: In strict compliance with the Data Privacy Act of 2012 (Republic Act No. 10173), all computer vision inference and coordinate extraction execute exclusively within the client device's browser memory (RAM) via WebAssembly. No live video streams, photograph captures, or biometric identifiers are saved, recorded, or transmitted to any external server or third-party cloud database. Once the browser tab is closed, all optical access terminates immediately.
- **Informed Participant Consent:** All thirty (30) usability study participants will receive a formal informed consent document detailing the research objectives, voluntary participation, and right to withdraw without academic penalty. No personally identifiable student records will be published.
- **Environmental Sustainability (SDG Alignment):** By converting existing, pre-installed laptop webcams into functional remote controllers via web software, the project directly operationalizes UN SDG 9 (Industry, Innovation, and Infrastructure) and UN SDG 12 (Responsible Consumption and Production), actively eliminating the electronic waste generated by manufacturing, packaging, and discarding single-use plastic Bluetooth clickers and disposable alkaline batteries.

4. INITIAL WEB SYSTEM FEATURES & EVALUATION PREP

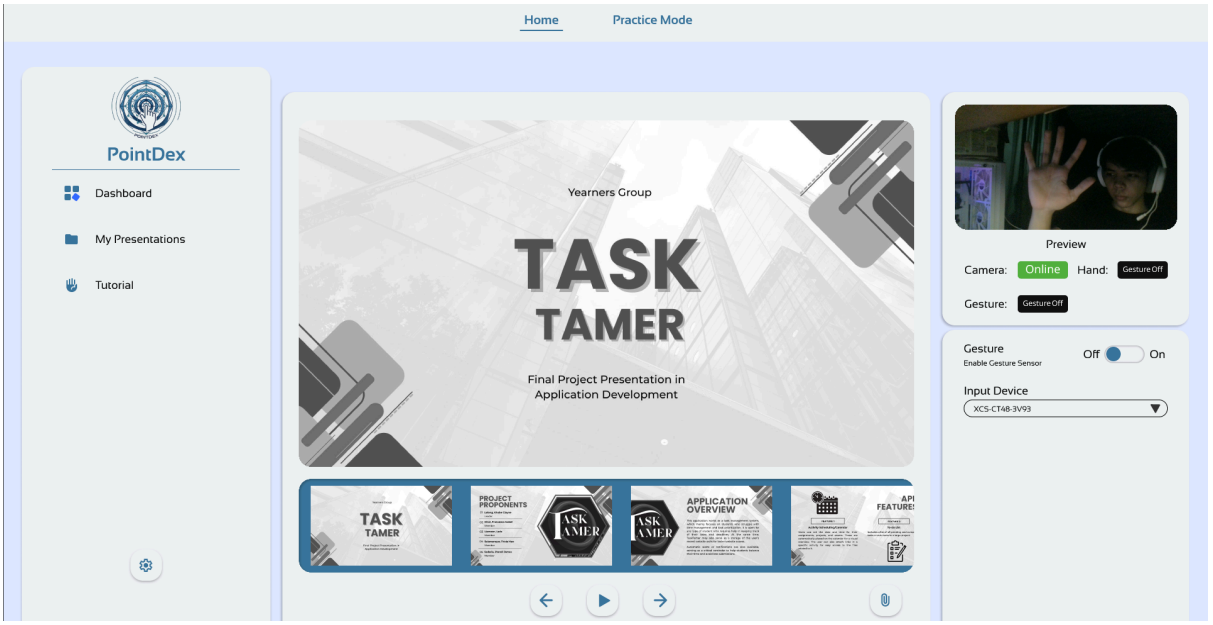


Figure 4.1: Proposed User Interface and Workflow

4.1 Document Core Event-Driven Features

Real-Time Notifications / WebSocket Streaming

- **Live Webcam Feeds & Overlays:** The system constantly streams camera video in real-time, drawing hand tracking points instantly on the screen as the presenter moves.
- **Instant Status Alerts:** Visual alert badges pop up immediately on screen if the webcam disconnects, if room lighting drops too low, or if camera permissions are blocked.
- **Unrecognized Gesture Feedback**: If the user performs an unclear movement, a quick pop-up message (e.g., "Gesture not recognized") appears to notify the presenter without interrupting the presentation.

Asynchronous Event Handlers & State Management

- **Gesture-to-Slide Control:** When the user swipes right, an event listener instantly catches the movement and sends a simulated "Right Arrow" keystroke to turn the slide.
- **Live Settings Sliders:** As the presenter adjusts sliders for Gesture Sensitivity or Command Cooldown, the system updates its settings in real time without refreshing the page.
- **Cooldown Prevention:** To prevent skipping multiple slides at once, a timer locks new inputs for 1.5 seconds immediately after a gesture is confirmed.

Background Event Processing

- **Gesture Smoothing Queue:** The web app buffers raw hand coordinate points across consecutive camera frames to ensure accidental hand twitches don't trigger slide changes.
- **Action Rate-Limiting:** Confirmed gesture events are filtered through a background cooldown timer before triggering any presentation keystroke.
- **Local Debug Logger:** System events, confidence scores, and slide triggers are recorded silently in memory to assist with QA testing during presentations.

4.2 Prepare Feature Metrics & Test Scenarios

Metric	Goal	How It Is Tested
Response Time (Latency)	**Under 100 milliseconds**	**Time measured from hand swipe completion to slide change.**
Frame Rate (FPS)	**At least 30 FPS**	**Webcam tracking smoothness during live movement.**
Accuracy Rate	**95% or higher**	**Successful slide triggers across 100 test swipes per gesture.**
False Triggers	**Less than 2%**	**Monitoring unintended slide changes during normal talking/gesturing.**

Black-Box Test Cases

1. **TC-01: Swipe Right to Advance Slide**
   - **Action:** Presenter performs a clear swipe right to gesture in front of the webcam.
   - **Expected Result:** The system detects the gesture, displays a visual confirmation indicator, and advances to the next slide.

2. **TC-02: Cooldown Double-Trigger Prevention**
   - **Action:** Presenter swipes right twice in rapid succession (within 0.5 seconds).
   - **Expected Result:** The first swipe changes the slide; the second swipe is ignored by the cooldown timer, preventing double-slide jumps.

3. **TC-03: Camera Disconnection Handling**
   - **Action:** Disconnect the webcam while the presentation controller is running.
   - **Expected Result:** The system displays a clear banner notification without freezing or crashing the browser page.

System Usability Scale (SUS) Evaluation Plan

A standard 10-statement questionnaire given to student and testers after trying the system:

1. I would like to use this gesture presentation tool frequently.
2. I found the setup process unnecessarily complicated.
3. The system was easy to use without a physical clicker or mouse.
4. I would need technical support to use this system.
5. The gesture detection and slide controls felt well integrated.
6. There was too much inconsistency between my hand movements and slide turns.
7. Most presenters would learn to use this system quickly.
8. The system felt awkward or tiring to use during a pitch.
9. I felt confident navigating slides using hand gestures.
10. I needed to learn a lot before I could start using the system.